

Embedded Systems Project Roadmap

Your Name

September 16, 2024

1 Phase 1: Foundation and System Understanding

1. Clocks and Timers: Understanding System Pulse

- *Objective:* Understand how clocks and timers drive the microcontroller and affect system timing.
- *Data Source:* System clock, SysTick, and basic timers (TIM).
- *Data Structures:* Circular Queue.
- *What to Learn:* Basic timer setup, understanding clock prescalers, timers' role in periodic interrupts, and how these affect real-time tasks.
- *Testable Outcome:* Configure timers to generate periodic interrupts and verify timing accuracy using interrupt handlers and SysTick for precise delays.

2. Basic Peripherals (UART, SPI, I2C, GPIO)

- *Objective:* Master the initialization and use of basic peripherals for data communication and control.
- *Data Source:* UART, SPI, I2C peripherals, GPIO pins.
- *Data Structures:* Queue (for data buffering).
- *What to Learn:* Understand how to configure and communicate using peripherals such as UART for serial communication, SPI for high-speed data transfer, I2C for inter-chip communication, and GPIO for digital I/O.
- *Testable Outcome:* Set up data communication using each peripheral, transmit and receive data, and control LEDs or external sensors via GPIO.

3. PWM (Pulse Width Modulation)

- *Objective:* Learn to generate PWM signals for controlling peripherals like motors or LEDs.

- *Data Source*: Timers configured for PWM mode.
- *What to Learn*: Configure timers to produce PWM signals and control duty cycles.
- *Testable Outcome*: Control LED brightness or motor speed using PWM.

4. Low Power Modes

- *Objective*: Learn to implement power-saving strategies in embedded systems.
- *Data Source*: Power control unit (PCU).
- *What to Learn*: Understand the different power modes (sleep, stop, standby) and how to wake the microcontroller from low-power states using interrupts.
- *Testable Outcome*: Demonstrate power-saving modes and configure the system to wake up using a timer or external interrupt.

5. Interrupt-Driven Task Scheduling

- *Objective*: Learn how interrupts drive real-time scheduling in embedded systems.
- *Data Source*: Timer interrupts.
- *Structures*: Priority Queue, Doubly Linked List.
- *Algorithms*: Round-Robin Scheduling, Earliest Deadline First (EDF).
- *What to Learn*: Implement a basic task scheduler using timers and interrupts.
- *Testable Outcome*: Implement an interrupt-driven scheduler, measure interrupt latency, and showcase task switching using a priority queue.

6. External Interrupts and Buttons

- *Objective*: Learn how to handle external interrupts triggered by buttons or other input devices.
- *Data Source*: GPIO configured for external interrupts.
- *What to Learn*: Configure external interrupts and implement debouncing strategies.
- *Testable Outcome*: Create a button-triggered event using external interrupts.

2 Phase 2: Intermediate Embedded Tasks and Data Structures

1. Temperature and Voltage Monitoring

- *Objective:* Develop real-time sensor monitoring using internal temperature sensors and voltage references.
- *Data Source:* Internal temperature sensor, voltage reference (VREFINT).
- *Data Structures:* Ring Buffer, Priority Queue.
- *Algorithms:* Moving Average, Exponential Smoothing, Threshold Detection.
- *What to Learn:* Understand how to read ADC values, apply smoothing algorithms, and detect threshold conditions.
- *Testable Outcome:* Create a monitoring system that logs temperature and voltage, applying a moving average filter to smooth sensor data.

2. Real-Time Clock (RTC) and Timestamps

- *Objective:* Implement real-time clock functionality for logging and scheduling purposes.
- *Data Source:* RTC, SysTick timer.
- *Data Structures:* Binary Search Tree.
- *Algorithms:* Scheduling algorithms, Binary Search.
- *What to Learn:* Understand how RTCs work in conjunction with system clocks for timekeeping and task scheduling.
- *Testable Outcome:* Set up the RTC to provide timestamps for event logging and use binary search trees to manage scheduled tasks.

3. Watchdog Timers for System Stability

- *Objective:* Use watchdog timers to monitor system stability and recovery from faults.
- *Data Source:* Independent and Window Watchdog Timers (IWDG, WWDG).
- *Data Structures:* Circular Queue.
- *Algorithms:* Timeout Management, Monitoring and Alerting.
- *What to Learn:* Understand how watchdog timers prevent system lockups and how to manage timeouts.
- *Testable Outcome:* Implement a watchdog system that monitors key tasks, triggers resets, and logs recovery data.

3 Phase 3: Signal Processing and Advanced Data Streaming

1. Real-Time Signal Processing with ADC

- *Objective:* Learn to handle high-speed analog data, perform basic signal processing, and apply filters.
- *Data Source:* Analog-to-Digital Converter (ADC).
- *Data Structures:* Circular Queue, FFT Buffer.
- *Algorithms:* FFT, Sliding Window, Finite State Machine (FSM) for pattern recognition.
- *What to Learn:* Set up real-time ADC data acquisition and implement an FFT for basic signal analysis.
- *Testable Outcome:* Process real-time sensor data, perform FFT, and detect signal patterns.

2. SPI/I2C for External Sensors

- *Objective:* Interface with external sensors (e.g., accelerometers, temperature sensors) via SPI or I2C.
- *Data Source:* External sensor via SPI/I2C.
- *What to Learn:* Set up communication with external sensors, read sensor data, and process it in real time.
- *Testable Outcome:* Interface with an external sensor and use the data for real-time processing.

3. Fault Monitoring and Recovery

- *Objective:* Detect and recover from system faults using interrupts and NMI.
- *Data Source:* Non-Maskable Interrupt (NMI), system faults.
- *Data Structures:* HashMap, Circular Queue.
- *Algorithms:* Error Detection (Hamming Code), CRC.
- *What to Learn:* Handle critical system faults using non-maskable interrupts, detect errors, and implement recovery.
- *Testable Outcome:* Design a fault-tolerant system that logs errors and recovers gracefully.

4 Phase 4: Advanced Projects for Embedded Expertise

1. DMA for Data Streaming

- *Objective:* Use Direct Memory Access (DMA) to offload data streaming tasks from the CPU.
- *Data Source:* DMA controller, ADC.
- *Data Structures:* Ring Buffer.
- *Algorithms:* Circular Buffer Management.
- *What to Learn:* Configure DMA for continuous data transfer and implement efficient data streaming algorithms.
- *Testable Outcome:* Build a real-time data streaming system using DMA and verify performance gains compared to CPU-managed data transfer.

2. Memory Management and Flash Programming

- *Objective:* Implement robust memory management techniques and work with flash memory.
- *Data Source:* Flash memory, EEPROM.
- *Data Structures:* HashMap, Linked List.
- *Algorithms:* Wear-Leveling, CRC for Data Integrity.
- *What to Learn:* Manage non-volatile memory, handle wear leveling, and ensure data integrity using CRC.
- *Testable Outcome:* Implement a memory manager that efficiently handles flash writes and data retention.

3. Power Consumption Monitoring

- *Objective:* Monitor and optimize power consumption in embedded systems.
- *Data Source:* Internal power sensors.
- *Data Structures:* Priority Queue, HashMap.
- *Algorithms:* Power Optimization (Dynamic Voltage/Frequency Scaling), Energy Profiling.
- *What to Learn:* Implement dynamic power management and measure the energy profile of embedded applications.
- *Testable Outcome:* Create a system that dynamically scales power usage based on task priority and workload.

4. Bootloader Development

- *Objective:* Implement a bootloader for firmware updates over UART or USB.
- *Data Source:* Flash memory.
- *What to Learn:* Understand the principles of bootloading and flash memory management.
- *Testable Outcome:* Create a basic bootloader that allows firmware updates and implements basic error handling during flashing.

5 Phase 5: CMSIS Automation for Testing and Validation

1. CMSIS Automation Testing Suite (Linux)

- *Objective:* Automate embedded system testing using CMSIS and Linux toolchains.
- *Tools:* CMSIS, StdPeriph automation suite.
- *Data Structures:* Custom test harness, Linked List.
- *What to Learn:* Build an automated testing framework using CMSIS and demonstrate proficiency in embedded systems validation.
- *Testable Outcome:* Develop a comprehensive test suite that runs automated tests for embedded peripherals and system stability on STM32. Integrate CI/CD for continuous testing.